Chapter 28 — Secure Logging & PII Redaction

> **Part III — Authentication & Security** · Complexity: **Standard** (≥25 interview questions)
>
> **Prerequisites:** Ch 1 (networking/correlation IDs), Ch 19 (tokens — what must never be logged), Ch 6 (UI state — what not to dump). Defines the **logging contract** reused by Ch 82 (analytics), Ch 83 (crash reporting), Ch 84 (tracing).
>
> **Reference surface in StreamCart:** logging across Shop/Watch/Wallet for debugging and telemetry **without** leaking tokens, card numbers, emails, or other PII into Logcat, crash reports, or analytics.

28.1 Problem Definition and Real-World Context

Logging is essential for debugging and observability, but it's also one of the most common ways apps **leak sensitive data**. Logs flow to places you don't fully control — Logcat (readable by other apps' tooling on some devices, by `adb`, and by anyone with the device), crash reporters, analytics pipelines, and third-party SDKs. Naive logging causes real incidents:

- **Secrets in Logcat.** Logging a full request/response dumps `Authorization: Bearer …` tokens (Ch 19), session cookies, and card numbers — readable via `adb logcat` and, on older/rooted devices, by other apps. A token in a log is a token leaked.
- **PII in crash reports / analytics.** A crash breadcrumb or event carrying an email, address, or full `UiState` ships PII to third-party services (Crashlytics, analytics) — a privacy/compliance violation (GDPR/CCPA) even if "just for debugging."
- **Verbose logging shipped to release.** Debug-level logs left on in production leak internals and PII and hurt performance/battery.
- **Unstructured, unsearchable logs.** `Log.d(TAG, "stuff: $obj")` is useless at scale — no structure, no correlation, no redaction, no levels.

The architecture: a **centralized logging abstraction** (not raw `Log` / `println`) with **log levels**, **structured events**, **mandatory PII/secret redaction**, **release-vs-debug behavior** (verbose only in debug; no sensitive logs ever), **correlation IDs** (Ch 1/84), and **consent/privacy** controls. The core rule: **decide what is sensitive and never let it reach a log sink** — redaction is a design-time contract, not an afterthought.

Three practical use cases

1. **Networking (StreamCart core).** Log request/response metadata (method, path template, status, latency, correlation ID) but **redact** `Authorization`, bodies with PII, and query params that carry names/emails (Ch 1/10).
2. **Fintech (Wallet).** Never log card numbers, CVV, balances, or tokens; log transaction **ids** and outcomes only; strict release logging.
3. **Crash reporting (all).** Breadcrumbs carry state **types** and ids, never user content; crash reports are scrubbed of PII before upload (Ch 83).

Why this is architectural

Logging defines the **sensitive-data boundary** (what may leave the app via logs), the **structure/levels** that make logs useful, the **release-vs-debug** policy, and the **privacy/consent** model — cross-cutting properties that determine whether debugging leaks user data.

28.2 Requirements Engineering

Functional requirements

ID	Requirement
F1	Route all logging through a central abstraction (no raw `Log` / `println`).
F2	Support log levels; verbose/debug only in debug builds.
F3	**Redact** secrets (tokens) and PII (emails, cards, names) from all log output.
F4	Emit **structured** events (key/value) with correlation IDs (Ch 1/84).
F5	Never send PII/secrets to crash reporters or analytics.
F6	Respect user consent / privacy settings for telemetry.

Non-functional requirements

Attribute	Policy
Security/Privacy	No secrets/PII in any log sink; compliant (GDPR/CCPA).
Usability	Structured, correlated, searchable; useful for debugging.
Performance	Cheap in release (no-op/minimal); no main-thread heavy work.
Reliability	Redaction can't be forgotten (centralized + typed).
Maintainability	One logging contract; pluggable sinks.

Scope

- **In scope:** logging abstraction (Timber-style), levels, release vs debug, PII/secret redaction (structured + patterns), correlation IDs, crash/analytics hygiene, consent, what not to log.
- **Out of scope:** analytics pipeline design (Ch 82), crash reporting internals (Ch 83), distributed tracing (Ch 84), network interceptors (Ch 1).

Assumptions & constraints

- A central `Logger` interface (introduced in Ch 1); sinks for Logcat (debug), crash reporter, analytics. Timber as a common base. `minSdk 24`. **Redaction is mandatory and centralized.**

28.3 System Design Fundamentals

- **Centralized logging abstraction.** All code logs through one `Logger` interface (or Timber), never `android.util.Log` / `println` directly. This gives one place to enforce redaction, levels, structure, and sink routing — and lets you strip/relax logs per build.
- **Log levels + build gating.** Verbose/Debug for development only; Info/Warn/Error in release but **still redacted**. In release, the debug sink (Logcat) is a no-op or minimal; only Warn/Error with scrubbed content reach remote sinks.
- **Redaction, two complementary approaches:**
  - **Structured + typed (preferred):** log key/value attributes where sensitive values are wrapped in a `Redacted` / `Sensitive` type whose `toString()` masks them (`"*****"`), so you *can't accidentally* print the raw value. The schema marks which fields are sensitive.
  - **Pattern-based (defense-in-depth):** a redaction filter scrubs free-text messages for known patterns (emails, card numbers, bearer tokens) before they reach any sink — a backstop for stringly logs.
- **Allow-list over block-list.** Prefer logging an **explicit allow-list** of safe fields (ids, types, counts, status codes) rather than "log everything then try to remove PII" — you can't enumerate all PII, so default to *not* logging data values.
- **Correlation IDs (Ch 1/84).** A per-request/per-session id threaded through logs ties events together for debugging without needing user identifiers — structure that replaces logging PII for correlation.
- **Crash/analytics hygiene.** Crash breadcrumbs log **types and ids** (e.g., "state=Cart.Error", "orderId=…"), never user content; events are schema-validated to exclude PII; redaction applies before upload (Ch 82/83).
- **Consent/privacy.** Honor the user's telemetry consent (GDPR/CCPA); drop or anonymize logs/analytics for opted-out users; data-minimization by default.

Core insight: **route everything through one logger that enforces redaction by default (typed-sensitive + pattern backstop), gate verbosity by build, correlate with ids instead of PII, and keep sensitive data out of crash/analytics sinks.** The safest log is the one that never contains the secret.

28.4 High-Level Architecture


28.4 High-Level Architecture

Components & ownership

- **`Logger`** is the single entry point; raw `Log` / `println` are forbidden (lint).
- **Redaction** (typed + pattern) runs before any sink — can't be bypassed.
- **Level/build gating** routes to sinks (Logcat debug-only; crash/analytics scrubbed).
- **Correlation IDs** structure logs without PII.
- **Consent** controls telemetry sinks.

Direction: code → Logger → redact → level/build route → sink (all scrubbed).

28.5 Detailed Component Design



Component	Responsibility	Note
`Logger` / `LogEvent`	structured logging contract (Ch 1)	level, message, correlationId, attributes
`Sensitive<T>`	wrap values so `toString()` masks	typed, compile-time-safe redaction
redacting tree/filter	scrub patterns; route by build	backstop for free text
sinks	Logcat (debug) / crash / analytics	release: no verbose; scrubbed
consent gate	respect privacy settings	drop/anonymize telemetry

28.6 End-to-End Data Flow

A network log, redacted



Crash breadcrumb hygiene



28.7 Complete Kotlin Implementation

Package: `com.streamcart.core.logging`.

28.7.1 Structured logging contract (extends Ch 1)

```
enum class Level { VERBOSE, DEBUG, INFO, WARN, ERROR }

@JvmInline
value class Sensitive<T>(val value: T) {        // typed redaction: you can't print the raw
value
    override fun toString(): String = "****"      // masked everywhere by default
    fun reveal(): T = value                       // explicit, rare, never logged
}

data class LogEvent(
    val level: Level,
    val tag: String,
    val message: String,
    val correlationId: String? = null,            // ties events (Ch 1/84) without PII
    val attributes: Map<String, Any?> = emptyMap(),
    val throwable: Throwable? = null,
)

interface Logger { fun log(event: LogEvent) }
```

28.7.2 Redaction (typed + pattern backstop)

```
object Redactor {
    private val patterns = listOf(
        Regex("""Bearer\s+[A-Za-z0-9\-._~+/]+=*""") to "Bearer ****",     // tokens
        Regex("""\b[\w.%+-]+@[\w.-]+\.[A-Za-z]{2,}\b""") to "***@***",     // emails
        Regex("""\b(?:\d[ -]*?){13,19}\b""") to "****-****",               // card-like numbers
    )
    fun scrub(text: String): String = patterns.fold(text) { acc, (re, repl) -> acc.replace(re,
repl) }

    // Attributes: Sensitive<*> masks itself; also allow-list which keys may carry values.
    fun scrub(attrs: Map<String, Any?>): Map<String, Any?> = attrs.mapValues { (_, v) ->
        when (v) {
            is Sensitive<*> -> "****"
            is String -> scrub(v)
            else -> v
        }
    }
}
```

28.7.3 Redacting Timber tree (central sink routing by build)

```
class RedactingTree(private val crash: CrashReporter, private val isDebug: Boolean) :
Timber.Tree() {
    override fun log(priority: Int, tag: String?, message: String, t: Throwable?) {
        val safe = Redactor.scrub(message)                    // backstop scrub
        if (isDebug) Log.println(priority, tag, safe)         // Logcat: debug builds only
        if (priority >= Log.WARN) crash.log(safe)             // remote: scrubbed warn/error
only
        if (priority >= Log.ERROR) t?.let { crash.recordException(scrubStack(it)) }
    }
}

// Application:
if (BuildConfig.DEBUG) Timber.plant(RedactingTree(crashReporter, isDebug = true))
else Timber.plant(RedactingTree(crashReporter, isDebug = false))   // no verbose/debug to Logcat
```

28.7.4 Usage: allow-list structured logging (no PII)

```
// GOOD: ids, types, counts, status — no data values.
logger.log(LogEvent(
    level = Level.INFO, tag = "Checkout", message = "order placed",
    correlationId = correlationId,
    attributes = mapOf(
        "orderId" to order.id,              // id, not contents
        "itemCount" to order.items.size,
        "status" to "success",
        "token" to Sensitive(accessToken),  // masked -> ****
    ),
))

// BAD (never):
// Log.d("Checkout", "placing order for ${user.email}, card ${card.pan}, token $accessToken")
```

28.7.5 Crash breadcrumb + analytics hygiene

```
fun breadcrumb(state: UiState) {
    crash.log("screen_state=${state::class.simpleName}")      // TYPE, not contents
}
// Analytics (Ch 82): schema-validated event names/params; a lint/CI check rejects PII-shaped
params.
```

28.7.6 Networking redaction (ties to Ch 1)

```
// From Ch 1: HttpLoggingInterceptor only in debug + redact sensitive headers.
HttpLoggingInterceptor().apply {
    level = if (BuildConfig.DEBUG) HttpLoggingInterceptor.Level.BODY else
HttpLoggingInterceptor.Level.NONE
    redactHeader("Authorization"); redactHeader("Cookie"); redactHeader("Set-Cookie")
}
```

28.7.7 Module structure

```
:core:logging   Logger, LogEvent, Sensitive, Redactor, RedactingTree, consent gate
:core:network   HttpLoggingInterceptor config (Ch 1)
# Lint rule: forbid android.util.Log / println outside :core:logging.
```

28.8 Concurrency and Race Conditions

Scenario	Risk	Handling
High-volume logging	main-thread jank	async sink (dispatch off main); cheap no-op in release
Redaction bypassed via raw `Log`	leak	lint rule forbids raw `Log` / `println`; central logger only
Sensitive value reaches a sink pre-redaction	leak	typed `Sensitive` masks by default; pattern backstop
Concurrent correlation-id context	wrong id	pass id explicitly or via a coroutine context element
Crash during logging	lost/partial	logging must never throw; guard sinks

- **Redaction must be unbypassable:** enforce via a **typed** `Sensitive` wrapper (masks by default) **and** a lint rule banning raw `Log` / `println`, so a developer can't accidentally log a secret even by mistake.

28.9 Failure Scenarios and Recovery

Failure	Detection	Response	Outcome
Token/PII found in Logcat	security review / scan	fix call site; add to redaction; lint	leak closed
PII in a crash report	report review	scrub breadcrumbs/keys; data-minimize; re-upload policy	compliant
Verbose logs in release	build audit	gate by `BuildConfig.DEBUG`; no-op release sink	no leak/perf hit
Raw `Log` used	lint/CI	fail the build; route through `Logger`	enforced
Analytics event carries PII	schema check	reject param; redact; CI gate	compliant
Opted-out user logged	consent check	drop/anonymize telemetry	privacy honored
Unredacted free-text log	pattern backstop misses	prefer allow-list structured logging	minimized



> **Default to not logging data values.** Pattern redaction will miss some PII formats; an **allow-list of safe fields (ids/types/counts)** is the reliable design, with pattern scrubbing as a backstop.

28.10 Scaling the Implementation

- **Level 1.** Central `Logger`/Timber; verbose only in debug; redact `Authorization`.
- **Level 2.** Typed `Sensitive` + pattern redaction; allow-list structured logging; correlation IDs (Ch 1).
- **Level 3 — production.** Scrubbed crash breadcrumbs (types/ids); analytics schema validation (no PII); consent gating; lint banning raw `Log`.
- **Level 4 — large.** Remote log levels (adjust verbosity per release via remote config, Ch 41); sampling; log schema registry; automated PII scanning of logs/ events in CI.
- **Level 5 — enterprise.** DLP on telemetry pipelines, data-residency/retention policies, audited access to logs, differential-privacy/anonymized analytics, compliance (GDPR/CCPA/HIPAA) tooling.

Avoid overengineering: most apps need a central redacting logger + allow-list structured logging + consent. Log schema registries and DLP are for large/regulated orgs.

28.11 Alternative Approaches and Trade-offs

Dimension	Option A	Option B	When
Redaction	Typed `Sensitive` (allow-list)	pattern block-list only	Typed/allow-list is reliable; patterns are a backstop, not sufficient alone
Logger	Central abstraction (Timber)	raw `android.util.Log`	Central always (redaction/levels/sinks); raw is unmanageable
Release logging	Warn/Error scrubbed, no verbose	full logging	Minimal scrubbed in release; full only in debug
Correlation	Correlation IDs	log user identifiers	IDs (no PII) for correlation
Crash breadcrumbs	Types + ids	raw state/content	Types/ids only; never content
Analytics	Schema-validated (no PII)	free-form params	Validated; free-form leaks PII

Allow-list vs block-list redaction: a block-list (scrub known PII patterns) will inevitably miss formats (localized addresses, new token shapes) — you can't enumerate all sensitive data. An **allow-list** (log only explicitly-safe fields: ids, types, counts, status) defaults to safety; use pattern scrubbing as a defense-in-depth backstop for any free-text that slips through. Prefer allow-list.

Central logger vs raw Log: a central abstraction lets you enforce redaction, switch verbosity by build, route to multiple sinks, add correlation, and lint-ban raw logging. Raw `Log.d` scattered everywhere makes redaction impossible to guarantee — the first place secrets leak.

28.12 Security & Privacy Considerations

- **Never log secrets or PII** — tokens (Ch 19), passwords, card/CVV, full `Authorization`, cookies, emails, names, addresses, precise location, full request/response bodies. Redact at the source.
- **Logcat is not private** — readable via `adb`, by some tooling, and on rooted/old devices by other apps; treat anything logged as potentially exposed.
- **Crash reporters and analytics are third parties** — scrubbing must happen **before** data leaves the app; log types/ids, not content.
- **Allow-list** safe fields; typed `Sensitive` + lint make redaction unbypassable.
- **Honor consent** (GDPR/CCPA): drop/anonymize telemetry for opted-out users; minimize data; define retention.
- **Correlation IDs** enable debugging without logging user identifiers.
- **Redact stack traces/exception messages** too (they can contain PII/secrets).
- Client vs backend: redaction here protects **client-emitted** logs/telemetry; the backend has its own logging hygiene — but a token logged on the client is already leaked regardless of backend practices.

28.13 Performance and Resource Optimization

- **Release logging should be near-free:** gate verbose/debug out of release (no-op tree), so no string building/IO happens on hot paths.
- **Avoid eager string concatenation** — use lazy/structured logging so disabled logs cost nothing; don't build big messages you won't emit.
- **Async sinks** for remote logging (don't block the main thread); batch telemetry (Ch 14/82).
- **Redaction cost** is a few regex passes — keep patterns tight; prefer structured/ typed (no regex) on hot paths.
- **Sampling** high-volume logs/events to control cost.
- **Profiling:** logging overhead on hot paths, telemetry payload size, battery/ network from remote logging.

28.14 Testing Strategy

Test that redaction works and that forbidden APIs are banned.

```
class RedactorTest {
    @Test fun `masks bearer tokens`() {
        assertEquals("Authorization: Bearer ****",
            Redactor.scrub("Authorization: Bearer eyJhbGciOi.secret.sig"))
    }
    @Test fun `masks emails and card numbers`() {
        assertEquals("user ***@*** paid ****-****",
            Redactor.scrub("user ada@example.com paid 4111 1111 1111 1111"))
    }
    @Test fun `Sensitive never prints the raw value`() {
        assertEquals("****", Sensitive("sk_live_abc").toString())
        assertFalse(LogEvent(Level.INFO, "t", "x", attributes = mapOf("tok" to
Sensitive("abc")))
            .toString().contains("abc"))
    }
    @Test fun `release tree does not log verbose to Logcat`() {
        val tree = RedactingTree(FakeCrash(), isDebug = false)
        // assert verbose/debug are no-ops; warn/error go to crash scrubbed
    }
}
```

Also: a **lint/CI rule** test that fails on `android.util.Log` / `println` usage outside `:core:logging`; an analytics schema test rejecting PII-shaped params; a crash-breadcrumb test asserting only types/ids. Consider a CI job scanning logs/ test output for secret patterns.

28.15 Production Observability

- **Meta-metrics:** redaction-filter hit rate (how often secrets are caught — a hit means a call site tried to log one; fix it), log volume by level, telemetry payload size, consent opt-out rate, dropped-log rate.
- **What good logs look like:** structured events with correlation IDs, status, latency, ids, types — searchable, correlatable, PII-free (feeds Ch 82/84).
- **Debugging the logging itself:** token appears in a log → a raw `Log` call or missing redaction (add lint + pattern); PII in crash report → unscrubbed breadcrumb/key; can't correlate events → missing correlation ID (Ch 1/84).
- **Alerting:** CI/log-scan finds a secret pattern in shipped logs → security incident; redaction-hit spike → a new leaky call site.
- **Privacy:** periodic PII audits of logs/analytics; honor data-subject deletion.

28.16 System Design Interview Preparation

> ≥25 questions.

Fundamentals

1. *Why is logging a security risk?* Logs reach Logcat/crash/analytics/third parties; secrets/PII in them are leaked.
2. *Is Logcat private?* No — `adb`, tooling, and some apps/rooted devices can read it.
3. *Why a central logger?* One place to enforce redaction, levels, sinks, correlation; raw `Log` can't be governed.
4. *Allow-list vs block-list redaction?* Allow-list (log only safe fields) is reliable; pattern block-list is a backstop (misses formats).
5. *What should never be logged?* Tokens, passwords, card/CVV, `Authorization`, cookies, emails/names/addresses, precise location, full bodies.

Redaction

6. *Typed redaction?* Wrap sensitive values in a `Sensitive` type whose `toString()` masks — can't accidentally print raw.
7. *Pattern redaction?* Regex-scrub messages for tokens/emails/cards as a backstop.
8. *How make redaction unbypassable?* Typed `Sensitive` + lint banning raw `Log`/ `println`.
9. *Redact stack traces?* Yes — exceptions/messages can carry PII/secrets.

Levels/build

10. *Debug vs release logging?* Verbose/debug only in debug; release emits minimal scrubbed warn/error.
11. *Why gate verbose out of release?* Leaks + performance/battery.
12. *Remote log levels?* Adjust verbosity per release via remote config (Ch 41).

Correlation/structure

13. *How debug without logging PII?* Correlation IDs (Ch 1/84) + structured ids/types.
14. *Structured vs stringly logs?* Structured (key/value) is searchable, redactable, correlatable.

Crash/analytics

15. *Crash breadcrumbs?* Types + ids, never content; scrub before upload (Ch 83).
16. *Analytics PII?* Schema-validated params; reject PII-shaped values (Ch 82).
17. *Third-party SDKs?* Treat as external sinks; scrub before data reaches them.

Privacy/consent

18. *GDPR/CCPA?* Honor consent; drop/anonymize telemetry for opted-out; minimize + retention.
19. *Data-subject deletion?* Logs/analytics must support it (no permanent PII).

Performance

20. *Release logging cost?* Near-zero (no-op trees, lazy/structured; no eager concatenation).
21. *Remote logging?* Async + batched + sampled.

Testing/governance

22. *How test redaction?* Assert masks for tokens/emails/cards; `Sensitive` never prints raw.
23. *How enforce no raw Log?* Lint/CI rule failing the build.
24. *CI PII scanning?* Scan logs/events for secret/PII patterns.

Production debugging

25. *Token in a log?* Raw `Log` or missing redaction → fix + lint.
26. *PII in crash report?* Unscrubbed breadcrumb/key.
27. *Can't correlate?* Missing correlation ID.

Design-change challenges

1. **Audit reveals tokens in Logcat.** Central redacting logger + typed `Sensitive`
    - `redactHeader` + lint.
2. **PII shipping to Crashlytics.** Scrub breadcrumbs to types/ids; schema-validate.
3. **Comply with GDPR consent.** Consent gate on telemetry; anonymize; retention.
4. **Debug a prod issue without PII.** Correlation IDs + structured ids; raise log level via remote config.
5. **Prevent future leaks.** Lint ban raw `Log`; CI secret/PII scanning; allow-list logging.

Mock walkthrough (abridged)

Clarify (what's sensitive, sinks, compliance) → central `Logger`/Timber → allow-list structured logging + typed `Sensitive` + pattern backstop → levels gated by build → correlation IDs → scrub crash/analytics → consent → lint/CI enforcement → testing redaction.

28.17 Practical Exercises

1. **Beginner.** Add a central redacting logger that masks `Authorization` and only logs verbose in debug. *Eval:* no token in Logcat; release quiet.

2. **Intermediate.** Implement typed `Sensitive` + pattern redaction (tokens/emails/ cards); test masks. *Eval:* raw values never printed.
3. **Advanced.** Convert stringly logs to allow-list structured events with correlation IDs; scrub crash breadcrumbs to types/ids. *Eval:* searchable, PII-free, correlatable.
4. **Production debugging.** Fix an app logging emails/cards to Logcat and PII to Crashlytics. *Eval:* redacted; crash reports scrubbed.
5. **System design challenge.** Add a lint rule banning raw `Log` / `println`, CI PII scanning, and consent-gated telemetry. *Eval:* raw log fails build; opted-out users not tracked.

(Solutions withheld unless requested.)

28.18 Chapter Summary & Reference Sheet

Key decisions: route **all** logging through a **central abstraction** (ban raw `Log` / `println` via lint); **default to an allow-list** of safe fields (ids, types, counts, status) and **wrap sensitive values in a typed** `Sensitive` that masks by default, with **pattern scrubbing** as a backstop; **gate verbosity by build** (verbose/debug only in debug; minimal scrubbed warn/error in release); **correlate with IDs**, not PII (Ch 1/84); keep **secrets/PII out of crash reports and analytics** (types/ids only; scrub before upload); **honor consent** (GDPR/CCPA) and minimize data. **The safest log never contains the secret.**

Concepts: central logging, log levels, typed vs pattern redaction, allow-list vs block-list, structured logging, correlation IDs, crash/analytics hygiene, consent/ data-minimization.

Common mistakes: logging tokens/PII/full bodies; raw `Log` / `println` everywhere; block-list-only redaction (misses formats); verbose logs in release; PII in crash breadcrumbs/analytics; eager string building on hot paths; ignoring consent; unredacted stack traces.

Checklist:

- [ ] central logger (no raw `Log`)
- [ ] allow-list structured fields
- [ ] typed `Sensitive` + pattern backstop
- [ ] `redactHeader(Authorization/Cookie)`
- [ ] verbose debug-only
- [ ] scrubbed crash breadcrumbs (types/ids)
- [ ] analytics schema (no PII)
- [ ] correlation IDs
- [ ] consent-gated telemetry
- [ ] lint/CI enforcement.

Key APIs/tools: Timber (`Tree`), `android.util.Log` (banned outside core), `HttpLoggingInterceptor.redactHeader` (Ch 1), a `Sensitive<T>` value class, correlation IDs (Ch 1/84), lint custom rules, gitleaks/PII CI scanning, Crashlytics `setCustomKey` / `log` (scrubbed, Ch 83).

References:

- Timber: https://github.com/JakeWharton/timber

- OkHttp logging + redaction: https://square.github.io/okhttp/features/interceptors/
- OWASP logging cheat sheet: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
- Android privacy best practices: https://developer.android.com/privacy-and-security/security-tips
- (feeds) Ch 82 Analytics, Ch 83 Crash Reporting, Ch 84 Tracing

Chapter review (Phase 4)

- **Completeness:** all 18 sections; architecture/class/flow/failure diagrams; Logger + Sensitive + Redactor + RedactingTree + allow-list usage + crash/analytics hygiene; concurrency (unbypassable redaction); failure (leak → fix+lint); 5 levels; allow-vs-block and central-vs-raw trade-offs; privacy/consent; perf (near-free release); redaction + lint tests; observability (redaction-hit metric); 27 Q&A + 5 challenges. ✅
- **Expand later:** analytics pipeline (Ch 82), crash reporting (Ch 83), tracing/ correlation IDs (Ch 84), DLP/compliance.
- **Interview coverage:** exceeds ≥25 bar. **Part III — Authentication & Security (Ch 19–28) is complete.**